09 — Deployment & Operations

1. Azure topology

Resource	Type	Purpose
`rg-Solo-Website`	Resource Group	Container for everything
Static Web App	`Microsoft.Web/staticSites`	Hosts React SPA + CDN
Container Apps Environment	`Microsoft.App/managedEnvironments`	Hosts the backend container
Container App	`Microsoft.App/containerApps`	NestJS API (`/api`)
Azure Container Registry	`Microsoft.ContainerRegistry/registries`	Stores backend images
User-assigned Managed Identity	`Microsoft.ManagedIdentity`	`AcrPull` to ACR
PostgreSQL Flexible Server	`Microsoft.DBforPostgreSQL/flexibleServers`	App database
Storage Account + `media` container	`Microsoft.Storage`	Image / file storage
Log Analytics Workspace	`Microsoft.OperationalInsights`	Log sink
Application Insights	`Microsoft.Insights/components`	App telemetry

All defined in `infra/main.bicep` with parameters in `infra/main.parameters.json` and orchestrated by `azd` (`azure.yaml`).

2. CI/CD with `azd`

```
# First-time / provisioning
azd auth login
azd env new prod                          # or azd env select <name>
azd env set AZURE_LOCATION eastus2
azd env set AZURE_RESOURCE_GROUP rg-Solo-Website
azd up                                     # provision + deploy

# Routine deploys
azd deploy backend                         # build, push to ACR, update ACA
azd deploy frontend                        # build dist + push to SWA
azd deploy                                 # both
```

What `azd deploy backend` does:

1. Reads `azure.yaml` → service `backend` (`./backend`, language `node`, host `containerapp`).
2. Builds the Docker image from `backend/Dockerfile`.
3. Tags and pushes to ACR.
4. Updates the Container App revision to the new image digest.
5. Rolls traffic to the new revision (default 100% → new).

Frontend deploy uses the SWA `swa-cli`-equivalent flow: `npm run build` in `frontend-react/`, then uploads `dist/` via SWA deployment token.

3. Required environment variables

Backend (Container App `secrets` + env vars):

Var	Notes
`DATABASE_URL`	`postgresql://user:pass@host:5432/db?sslmode=require`
`JWT_SECRET` / `JWT_REFRESH_SECRET`	≥ 32 random bytes each
`JWT_ACCESS_TTL` / `JWT_REFRESH_TTL`	e.g., `900s` / `30d`
`CORS_ORIGINS`	CSV of allowed origins
`STRIPE_SECRET_KEY` / `STRIPE_WEBHOOK_SECRET`	Stripe
`TABBY_*`, `TAMARA_*`	BNPL credentials
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`	Email
`BLOB_ACCOUNT`, `BLOB_CONTAINER`, `BLOB_KEY` (or use MI)	Media

Var	Notes
`APPINSIGHTS_CONNECTION_STRING`	Telemetry
`SWAGGER_ENABLED`	`false` in prod recommended
`THROTTLE_TTL`, `THROTTLE_LIMIT`	Optional override
`ADMIN_BOOTSTRAP_EMAIL` / `ADMIN_BOOTSTRAP_PASSWORD`	First-run seeding

Frontend (SWA app settings):

Var	Notes
`VITE_API_URL`	e.g., `https://backend-...azurecontainerapps.io/api`
`VITE_STRIPE_PUBLIC_KEY`	Stripe publishable key

4. Database operations

```
# From backend/
npx prisma migrate deploy   # in CI / startup
npx prisma generate         # build-time
npm run db:seed             # idempotent seeds
```

Manual SQL access: connect from the bastion or admin workstation IP allowed by the Postgres firewall rule.

Backups: Azure-managed, daily automated; point-in-time restore available. Pre-cleanup snapshot kept at `solo_ecommerce_pre_cleanup_20260208.dump`.

5. Observability

- **Logs:** Pino JSON logs go to stdout; Container Apps streams to Log Analytics. Query via Logs blade or `kusto` in App Insights.
- **Traces:** `backend/src/tracing.ts` initialises App Insights before Nest. Correlated trace IDs flow through HTTP requests.
- **Metrics:** ACA built-in CPU/memory/replicas; Postgres flexible server CPU, IOPS, connections.
- **Alerts** (recommended): server errors > 1% for 5 min, container restarts, Postgres CPU > 80% sustained, blob 5xx, Stripe webhook failures.

6. Routine operations

Task	Command
Roll new backend image	`azd deploy backend`
Roll back	Set ACA traffic to previous revision in portal
Restart container	Stop + start the active revision (portal)
Inspect live logs	`az containerapp logs show -n backend -g rg-Solo-Website --follow`
Scale replicas	`az containerapp update --name backend -g rg-Solo-Website --min-replicas 2 --max-replicas 10`
Apply DB migration	Bake into the container start script (`prisma migrate deploy && node dist/main.js`)

7. Smoke after deploy

Run the smoke test pack to validate auth + product + cart against the deployed backend:

```
cd backend
$env:SMOKE_API_URL = 'https://backend-qlyb5greec2io.whiteriver-ae956411.eastus2.azurec
$env:SMOKE_ADMIN_EMAIL = 'admin@solo-ecommerce.com'
$env:SMOKE_ADMIN_PASSWORD = '<secret>'
npm run test:smoke
node scripts/cleanup-smoke-data.js   # final sweep of any residual test data
```

End-to-end Playwright smoke from repo root:

```
npx playwright install chromium
npx playwright test e2e/smoke
```

8. Disaster recovery

- **DB lost** → restore PIT snapshot to a new flexible server, repoint `DATABASE_URL`, redeploy.
- **Container App lost** → `azd up` reprovisions; ACR holds the image digests for rollback.
- **SWA lost** → redeploy via `azd deploy frontend`; SWA tokens stored in the deployment env.
- **Region outage** → currently single-region; multi-region active- passive is a planned enhancement (geo-replicate ACR + Postgres + SWA).

9. On-call runbook (high level)

1. Page → check Azure portal alert + App Insights failure timeline.
2. Identify whether failure is API, DB, or third party (Stripe/Tabby).

3. If API — inspect logs, check recent deploy revision, roll back if regression.
4. If DB — check connections / CPU / IOPS in Postgres metrics.
5. If third party — check provider status; serve degraded checkout (disable that payment method via `site_settings`).
6. Post-mortem within 48h; capture root cause + corrective action.